Zenith inside MyKey

Goal: a small Zenith model running **offline on the phone** that makes MyKey's suggestions understand the whole sentence, not just the last word.

Languages, in priority order: English, Spanish, Japanese. Russian and Mandarin are kept as reserved slots in the tokenizer so they can be added later without starting over.

What Zenith does for the keyboard:

	English and Spanish	**Japanese**
Next word / phrase	"I'll see you" → *tomorrow / soon / there*	「今日は」→ *いい天気 / ありがとう*
While typing	"I'm going to the be" → *beach* over *best*	Picks the right kanji for the reading in context: かえる → 帰る after 家に, 蛙 after 池の
Autocorrect	"I want to by a car" → *buy*	(no autocorrect in kana input)
Later	Grey ghost-text completion of the sentence	Same

For Japanese the biggest win is **kanji choice**. MyKey already finds every possible conversion with the Mozc dictionary; Zenith decides which one fits the sentence. That's the part that makes Japanese keyboards feel smart, and the dictionary alone can't do it.

MyKey keeps its word lists, its Japanese dictionary and your learned words. Zenith only re-orders and adds on top. If the model is missing or slow, the keyboard falls back to what it does today.

Budget on the phone

Limit	**Target**
Model file	≤ 20 MB (int8)
Time per prediction	≤ 30 ms on the Xiaomi 14 Civi CPU
RAM	≤ 70 MB
Context	last 48–64 tokens

That means about **15–18M parameters**. That's a bit bigger than the English-only plan, because Japanese needs a much larger vocabulary (kana plus a few thousand kanji).

What to change in the Zenith repo

1. Data: chat-style text in three languages

TinyStories is the main blocker: the model only knows children's English stories. A keyboard needs the way people actually write messages.

Add `zenith/data/keyboard_corpus.py` that builds a mixed corpus:

Source	English	Spanish	Japanese	Why
OpenSubtitles monolingual (OPUS)	✓	✓	✓ (smaller)	Closest free thing to casual speech
Tatoeba sentences	✓	✓	✓	Short, clean everyday sentences
Wikipedia (first paragraphs)	✓	✓	✓	Names, places, wider vocabulary
CC-100 (filtered slice)			✓	More informal web Japanese, since Japanese subtitles are scarce
Your exported MyKey words (optional, stays local)	✓	✓	✓	Your style

Target mix of **tokens**: English 35%, Spanish 30%, Japanese 35%. Japanese gets a bigger share than its data size suggests, because its vocabulary is much broader.

Cleaning rules:

- Prefix each line with a language token: `<en>`, `<es>`, `<ja>`. MyKey always knows the current keyboard language, so it sends the tag and the model never drifts into the wrong language.
- Normalise with **NFKC**. For Japanese this turns half-width katakana and full-width ASCII into their standard forms, so the model doesn't learn three spellings of the same thing.
- Keep casing, accents (á é ñ ¿ ¡) and Japanese punctuation (、。「」).
- Drop lines over ~200 characters, URLs and lines that are mostly digits.
- Balance with temperature sampling so the smaller corpora aren't drowned.

Aim for **1–1.5B tokens** total. At the ~37k tokens/s you measured on the 3050, that's about 8–11 hours locally. You can also split it across two Kaggle sessions using the resume support you already built.

2. Tokenizer: switch to Unigram for Japanese

Your byte-level BPE splits on spaces first. Japanese has no spaces, so a whole sentence becomes one chunk and the merges come out poor. Change `train_tokenizer.py` to a **Unigram (SentencePiece-style)** model:

- `tokenizers.models.Unigram` with the `Metaspace` pre-tokenizer (`▁` marks word starts in English and Spanish, which MyKey needs to turn token predictions into whole words).
- `byte_fallback=True`, so rare kanji or emoji never become `<unk>`.

- `vocab_size` **32000**. About 8k is plenty for English plus Spanish, but Japanese needs the rest.
- NFKC normaliser built into the tokenizer, so training and the phone see identical text.
- Special tokens: `<pad> <bos> <eos> <unk> <en> <es> <ja>` plus reserved `<ru> <zh>`.

Check it before training: on held-out text, English and Spanish should average 1.2–1.5 tokens per word, and Japanese about 1–1.5 characters per token. If Japanese is mostly single characters, the vocabulary is too English-heavy, so rebalance the tokenizer training text.

3. Model config: `configs/zenith_keyboard.yaml`

```
model:
  vocab_size: 32000
  dim: 320
  n_layers: 7
  n_heads: 5
  n_kv_heads: 1        # one KV head keeps the phone-side cache tiny
  hidden_dim: 864
  max_seq_len: 64      # a keyboard only needs the recent sentence
  tie_embeddings: true
training:
  micro_batch_size: 256
  grad_accum_steps: 2
  total_steps: 40000
  max_lr: 1.0e-3
```

That's about 17.8M parameters: 10.2M in the tied embedding and 7.6M in the layers. It comes to roughly 18 MB at int8. Short sequences keep training fast.

4. Keyboard-specific evaluation

Perplexity alone won't tell you if it helps typing. Add `zenith/evaluation/keyboard_eval.py` with held-out chat sentences for **each language separately**:

English and Spanish

- **Next-word top-3 accuracy**: is the real next word in the top three? Target ≥ 35% (English), ≥ 30% (Spanish).
- **Prefix completion top-3**: given the first 1–3 letters, is the word in the top three? Target ≥ 70%.
- **Keystroke savings**: share of letters you don't have to type when tapping suggestions.

Japanese

- **Kanji choice accuracy**: the test that matters most. Use `fugashi` with UniDic to get the reading of each word in real Japanese sentences, look up all Mozc candidates for that reading, and check whether Zenith ranks the original kanji first given the sentence before it. Compare against Mozc's own ordering. Zenith should clearly beat it.

- **Next-phrase top-3**: is the next word (by the same segmentation) among the model's top three?

Run a plain bigram baseline for every metric too. Zenith has to win per language, not just on average.

5. Export for the phone

Add `zenith/export/`:

- `export_onnx.py`: export the forward pass with the KV cache as explicit inputs and outputs (fixed 64-token window), opset 17.
- `quantize.py`: int8 dynamic quantisation with `onnxruntime.quantization`. Keep the embedding table int8 too; it's most of the file.
- `parity_test.py`: same idea as your KV-cache check. Compare ONNX int8 with PyTorch fp32 on the same input: top-5 tokens should match on $\geq$ 95% of positions, **for each language**.
- `export_tokenizer.py`: write the Unigram pieces, scores and normaliser settings in a compact file MyKey can load without Python.

Two ways to run it on the phone. Pick one:

	ONNX Runtime Mobile	**Your own engine**
Effort	Low. MyKey adds one library (~5 MB)	High. Write RMSNorm, RoPE, GQA, SwiGLU in C (NDK) or Kotlin
Speed	Fast, uses NEON	As fast as you make it
Fits the from-scratch spirit	No	Yes, all the way to the phone

Start with ONNX to prove it helps, then replace it with your own engine if you want the whole stack to be yours.

6. What I build on the MyKey side

Once you send back `zenith_kb.onnx`, the tokenizer file and the eval numbers:

- `engine/ZenithModel.kt`: loads the model and tokenizer, keeps the KV cache between keystrokes, runs on a background thread and cancels stale requests.
- **English and Spanish**: score dictionary candidates by the model's probability of their tokens, constrained to the letters you've typed. Mix the scores: `a·zenith + b·word frequency + c·your habits`.
- **Japanese**: re-rank the Mozc candidate list by `P(sentence so far + candidate)`. Your own past kanji choices still come first. Next-phrase predictions appear when nothing is being typed.
- A **Smart predictions (Zenith)** switch in Settings → Autocorrect, with per-language on/off and a speed readout in Stats.

7. Personalisation, still private

Training on the phone is too heavy. Instead:

1. Export a MyKey backup (Settings → Backup). It's your own file, including your Japanese kanji choices.
2. Fine-tune a small LoRA adapter on your PC from it with a new `zenith finetune-personal` command.
3. Copy the adapter to the phone. MyKey loads it next to the base model.

Nothing touches the internet at any step.

Order of work

1. Corpus for English, Spanish and Japanese (step 1)
2. Unigram tokenizer, then check tokens per word for each language (step 2)
3. Train the 17.8M config and run the keyboard eval per language (steps 3–4)
4. If English next-word top-3 is $\geq$ 35% and Japanese kanji choice beats Mozc's order, export and run the parity test (step 5)
5. Send me the files and I wire it into MyKey (step 6)
6. Personal LoRA (step 7)
7. Later: add Russian and Mandarin using the reserved tokens